

TOPUP SYSTEM

API Technical Specification

Version 1.1 | 2025

Document Type	API Specification
Status	Draft
Audience	Integration Partners / Client Developers
Currency	DZD (Algerian Dinar) — single currency
Authentication	API Key per client account
Reconciliation	Client external_id supported on every transaction

1. Introduction

This document describes the technical specification for the TopUp System API. The API allows registered client applications to initiate mobile topup transactions, monitor transaction status, and manage their account balances programmatically.

Each client operates under an isolated account with its own API key, balance (in DZD), and transaction history. Clients may attach their own external_id to each transaction to facilitate reconciliation with their internal systems.

1.1 Key Capabilities

- Initiate topup transactions via a REST API
- Check real-time transaction status
- Balance reservation on topup creation (pre-authorization)
- Asynchronous processing via a managed job queue
- Single currency: DZD (Algerian Dinar)
- Client-supplied external_id for reconciliation on every transaction
- Per-client account isolation and API key authentication
- Transaction amount limits: min 100 DZD — max 10,000 DZD

2. System Architecture

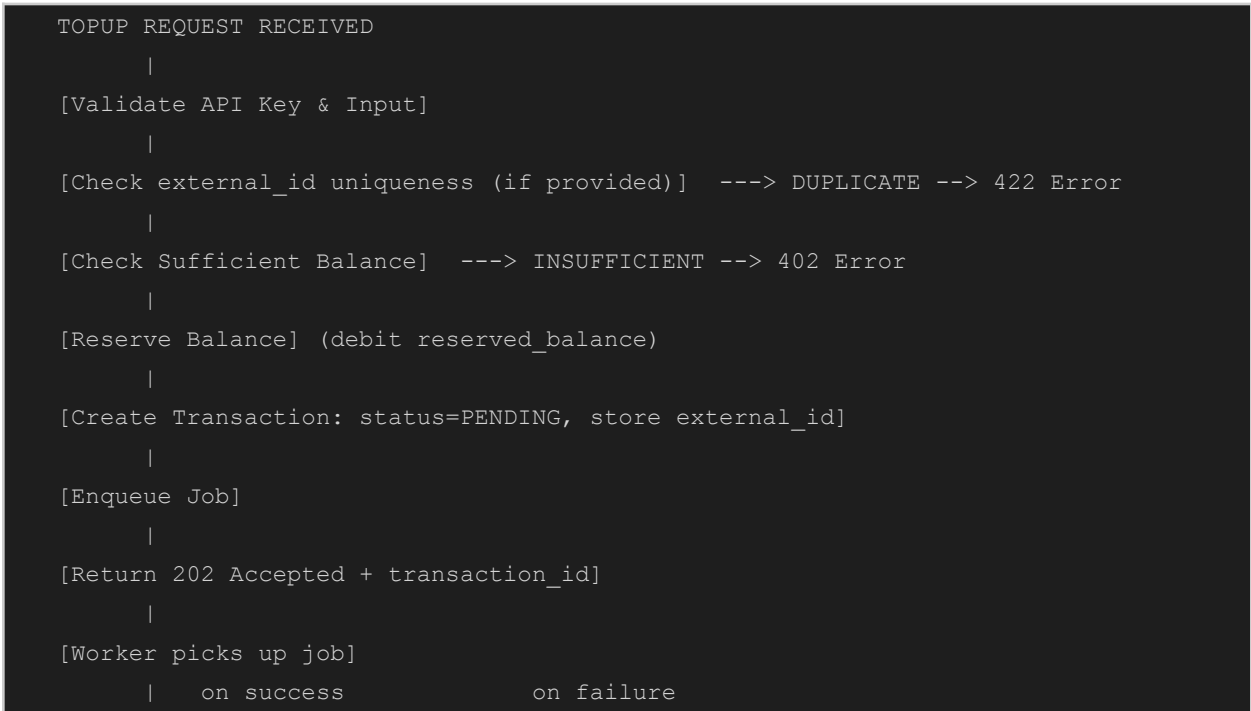
2.1 Overview

The TopUp System is built around four core components:

Component Overview		
API Gateway	Authenticates requests via API key, validates inputs, routes to business logic.	Stateless HTTP layer
Account Service	Manages per-client accounts: DZD balance and reservations.	Stateful, database-backed
Transaction Service	Creates, tracks and finalizes topup transactions, including external_id indexing.	Persists all state changes
Queue Worker	Processes topup jobs asynchronously; updates transaction status.	Async, retryable

2.2 Balance Reservation Flow

When a topup request is received and validated, the system immediately reserves the transaction amount from the client's available DZD balance. This prevents double-spending while the job is processed asynchronously.



```
[Finalize Transaction]      [Release Reservation]
[Debit available_balance]   [status=FAILED]
[status=COMPLETED]
```

2.3 Queue System

All topup operations are processed through an internal job queue. This ensures:

- Rate limiting and throttling per client or per operator
- Automatic retries on transient failures (configurable retry policy)
- Decoupling of the API response from the actual topup execution
- Ordered processing and auditability

Clients receive a `transaction_id` immediately on submission and must poll the status endpoint (or use webhooks, if configured) to track completion.

3. Authentication

Every API request must include the client's API key in the request header. API keys are issued per client account and must be kept confidential.

```
Authorization: ApiKey <your_api_key>
```

Example:

```
Authorization: ApiKey ak_live_a1b2c3d4e5f6...
```

API key rules:

- One active API key per client account (rotation supported)
- Keys are tied to a single account — cross-account usage is rejected
- Compromised keys must be revoked via the management console or support
- All requests are logged against the API key for auditing

4. Account Model

Each client has an account denominated exclusively in DZD (Algerian Dinar):

Account Balance Fields		
available_balance	Funds available for new topup transactions (DZD).	Decreases on reservation
reserved_balance	Funds locked for in-flight transactions (DZD).	Released on completion or failure
total_balance	Sum of available + reserved (DZD). Read-only.	Computed field
currency	Always DZD.	Fixed — single currency system

Balance reservation logic:

- On topup creation: `available_balance -= amount`; `reserved_balance += amount`
- On topup COMPLETED: `reserved_balance -= amount` (funds consumed)
- On topup FAILED/CANCELLED: `reserved_balance -= amount`; `available_balance += amount` (funds released)

5. Currency

The system operates exclusively in DZD (Algerian Dinar). All transaction amounts, balances, and API fields are expressed in DZD. There is no multi-currency or FX conversion layer.

Currency		
DZD	Algerian Dinar — sole system currency.	All amounts in DZD
Amount format	Integer — whole dinars only, no decimals.	e.g. 500, 1000, 10000
Min / Max	100 DZD minimum, 10,000 DZD maximum per transaction.	Enforced at API validation

6. Transaction Model

6.1 Transaction Fields

Transaction Object		
transaction_id	System-generated unique identifier (UUID v4).	Immutable

Transaction Object		
external_id	Client-supplied identifier for reconciliation.	Optional, unique per client, max 64 chars
client_id	Owning client account.	Derived from API key
msisdn	Target phone number in E.164 format.	Required, e.g. +2130600000000
amount	Topup amount in DZD.	Integer, 100 to 10,000
status	Current transaction status.	See status lifecycle below
created_at	ISO 8601 UTC timestamp of creation.	Set at creation
updated_at	ISO 8601 UTC timestamp of last update.	Updated on status change
completed_at	ISO 8601 UTC timestamp of terminal state.	Null until terminal
failure_reason	Error code/message if FAILED.	Null on success

6.2 external_id — Reconciliation Field

The `external_id` field allows clients to attach their own internal identifier (order ID, payment reference, job ID, etc.) to each topup transaction. This simplifies reconciliation between the client's system and the TopUp platform.

Rules:

- Optional on submission — if omitted, the field is null on the transaction
- Must be unique per client account — submitting a duplicate `external_id` returns a 422 error
- Maximum length: 64 characters. Allowed characters: alphanumeric, hyphens, underscores
- Immutable once set — cannot be changed after transaction creation
- Clients can look up a transaction by `external_id` via the GET /topups endpoint using the `?external_id=` query parameter

6.3 Transaction Status Lifecycle

Transaction Statuses		
PENDING	Transaction created, balance reserved, job queued.	Initial state
PROCESSING	Worker has picked up the job and is executing.	Transitional

Transaction Statuses		
COMPLETED	Topup delivered successfully.	Terminal — success
FAILED	Topup failed; reservation released.	Terminal — failure
CANCELLED	Cancelled before processing; reservation released.	Terminal

6.4 Amount Constraints

Constraint	Value	Notes
Minimum amount	100 DZD	
Maximum amount	10,000 DZD	
Currency	DZD only	
Decimal places	0 — whole dinars only	

7. API Endpoints

Base URL: <https://api.topup.example.com/v1>

7.1 Endpoint Summary

POST	/topups	Create a new topup transaction
GET	/topups/{transaction_id}	Retrieve transaction by system ID
GET	/topups?external_id={id}	Retrieve transaction by client external_id
GET	/topups	List all transactions for the client account
GET	/account	Get current account balance

7.2 Create Topup

POST /topups

Creates a new topup transaction. The amount is immediately reserved from the client's DZD balance and a job is enqueued for processing.

Request Body

```
{
  "msisdn":      "+2130600123456", // Required. E.164 phone number
  "amount":      500,                // Required. Integer DZD, 100-10000
  "external_id": "ORDER-8821"       // Optional. Client reconciliation ID
}
```

Success Response — 202 Accepted

```
{
  "transaction_id": "txn_f3alb2c4-...",
  "external_id":    "ORDER-8821",
  "status":        "PENDING",
  "msisdn":        "+2130600123456",
  "amount":        500,
  "currency":      "DZD",
  "reserved_at":   "2025-07-15T10:23:44Z"
}
```

Error Responses

POST /topups — Error Codes		
400 Bad Request	Missing or invalid fields (msisdn, amount out of range).	Fix request payload
401 Unauthorized	Missing or invalid API key.	Check Authorization header
402 Payment Required	Insufficient available balance.	Top up account first
422 Unprocessable	Duplicate external_id for this client account.	Use a unique external_id
429 Too Many Requests	Rate limit exceeded.	Retry after indicated delay
500 Server Error	Internal system error.	Contact support

7.3 Get Transaction by System ID

GET /topups/{transaction_id}

Retrieves the current status and full details of a transaction using the system-generated `transaction_id` returned at creation.

Success Response — 200 OK

```
{
  "transaction_id": "txn_f3a1b2c4-...",
  "external_id": "ORDER-8821",
  "status": "COMPLETED",
  "msisdn": "+2130600123456",
  "amount": 500,
  "currency": "DZD",
  "created_at": "2025-07-15T10:23:44Z",
  "updated_at": "2025-07-15T10:23:51Z",
  "completed_at": "2025-07-15T10:23:51Z",
  "failure_reason": null
}
```

7.4 Get Transaction by external_id

GET /topups?external_id={your_id}

Retrieves a transaction using the client-supplied `external_id`. This is the primary lookup endpoint for reconciliation workflows — allowing clients to query by their own internal order or payment identifiers.

Example Request

```
GET /v1/topups?external_id=ORDER-8821
Authorization: ApiKey ak_live_a1b2c3d4e5f6...
```

Returns the same transaction object as `GET /topups/{transaction_id}`. Returns 404 if no transaction with that `external_id` exists for the authenticated client.

7.5 Get Account

GET /account

Returns the current state of the authenticated client's account, including available and reserved DZD balances.

Success Response — 200 OK

```
{
```



```
"client_id":      "cli_abc123",
"currency":       "DZD",
"available_balance": 20850,
"reserved_balance": 1500,
"total_balance":  22350,
"updated_at":     "2025-07-15T10:20:00Z"
}
```

8. Standard Error Format

All API errors return a consistent JSON structure:

```
{
  "error": {
    "code":      "DUPLICATE_EXTERNAL_ID",
    "message":    "A transaction with external_id ORDER-8821 already exists.",
    "request_id": "req_9x8y7z..."
  }
}
```

Common Error Codes		
INVALID_AMOUNT	Amount is below 100 or above 10,000 DZD.	400
INVALID_MSISDN	Phone number is not valid E.164 format.	400
INVALID_EXTERNAL_ID	external_id exceeds 64 chars or has invalid characters.	400
INSUFFICIENT_BALANCE	Available DZD balance is too low.	402
DUPLICATE_EXTERNAL_ID	external_id already used by another transaction for this client.	422
UNAUTHORIZED	API key missing, invalid, or revoked.	401
NOT_FOUND	transaction_id or external_id not found for this client.	404
RATE_LIMITED	Too many requests. Retry after the Retry-After header value.	429

9. Integration Guidance

9.1 Reconciliation with external_id

The recommended reconciliation pattern is:

- Before submitting a topup, generate a unique external_id in your system (e.g. order ID, payment reference)
- Include external_id in the POST /topups request body
- Store the returned transaction_id alongside your external_id
- To look up a transaction later, use GET /topups?external_id={your_id} — no need to store or look up system UUIDs
- A 422 DUPLICATE_EXTERNAL_ID response confirms the transaction was already submitted — safe to look it up and check its status

9.2 Polling for Status

Since transactions are processed asynchronously, poll GET /topups/{transaction_id} or GET /topups?external_id={id} after submission:

- Poll after 2s, then every 3s, up to 10 attempts
- If still PENDING/PROCESSING after 30s, continue at 10s intervals
- COMPLETED and FAILED are terminal states — stop polling
- Implement exponential backoff on 429 responses

9.3 Multi-Client Isolation

Each API key is scoped to a single client account. external_id uniqueness is enforced per client — the same external_id value used by two different clients is allowed. A client cannot access another client's transactions or balance.

10. Security Requirements

- All API communication must use HTTPS (TLS 1.2+). HTTP is rejected.
- API keys must never be embedded in client-side code or version control.
- Rotate API keys immediately if compromise is suspected.
- All API requests are logged with timestamp, IP, client ID, and request ID.
- Rate limits are enforced per API key to prevent abuse.

11. Versioning & Support

The API is versioned via the URL path (/v1/...). Breaking changes will be introduced in new versions with a deprecation notice period of at minimum 90 days.

For integration support, contact: api-support@topup.example.com

End of Specification — TopUp System API v1.1